APM1110 Probability & Probability Distributions Formative Assessment 8

Angelina Jolie C. Peña

2026-11-06

1. An urn contains 10 red, 8 black, and 7 green balls. One of the colors is chosen at random (meaning that the chosen color is equally likely to be any of the 3 colors), and then 4 balls are randomly chosen from the urn.

Let X be the number of these balls that are of the chosen color.

Given:

Red = 10, Black = 8, Green = 7

Total = 25 balls

Number of draws = 4

Each color is equally likely to be selected.

(a) Find P(X = 0).

Using the hypergeometric distribution:

P(X = x) = [choose(m,x) * choose(N-m,n-x)] / choose(N,n)

Where:

N = 25

n = 4

x = 0

For RED:

P(X = 0 | Red)

= [choose(10,0) * choose(15,4)] / choose(25,4)

= [1 * 1365] / 12650

= 1365 / 12650

= 0.1079

```r
# Define the total number of balls and draws.

total_balls <- 25
draws <- 4

# Store the number of balls for each color.

color_counts <- c(Red = 10, Black = 8, Green = 7)

# (a) Calculate P(X = 0) using the hypergeometric distribution.
#
# dhyper(x, m, n, k):
# x = number of selected-color balls drawn
# m = number of balls of the selected color
# n = number of balls of other colors
# k = number of draws

prob_X0_given_color <- dhyper(
  x = 0,
  m = color_counts,
  n = total_balls - color_counts,
  k = draws
)

# Average the three conditional probabilities
# because each color has an equal probability of selection.

prob_X0 <- mean(prob_X0_given_color)

cat("PROBLEM 1(a)\n")
```

```
## PROBLEM 1(a)
```

```r
cat("P(X = 0 | Red) =", round(prob_X0_given_color[1], 4), "\n")
```

```
## P(X = 0 | Red) = 0.1079
```

```r
cat("P(X = 0 | Black) =", round(prob_X0_given_color[2], 4), "\n")
```

```
## P(X = 0 | Black) = 0.1881
```

```r
cat("P(X = 0 | Green) =", round(prob_X0_given_color[3], 4), "\n")
```

```
## P(X = 0 | Green) = 0.2419
```

```r
cat("P(X = 0) =", round(prob_X0, 4), "\n\n")
```

```
## P(X = 0) = 0.1793
```

```r
# (b) Calculate P(Xi = 1).

# Average the proportions of each color in the urn.

prob_Xi <- mean(color_counts / total_balls)

# Display the probability for all four positions.

cat("PROBLEM 1(b)\n")
```

```
## PROBLEM 1(b)
```

```r
cat("P(X1 = 1) =", round(prob_Xi, 4), "\n")
```

```
## P(X1 = 1) = 0.3333
```

```r
cat("P(X2 = 1) =", round(prob_Xi, 4), "\n")
```

```
## P(X2 = 1) = 0.3333
```

```r
cat("P(X3 = 1) =", round(prob_Xi, 4), "\n")
```

```
## P(X3 = 1) = 0.3333
```

```r
cat("P(X4 = 1) =", round(prob_Xi, 4), "\n\n")
```

```
## P(X4 = 1) = 0.3333
```

```r
# (c) Calculate E[X].

# By linearity of expectation:
# E[X] = 4 * P(Xi = 1)

expected_X <- draws * prob_Xi

cat("PROBLEM 1(c)\n")
```

```
## PROBLEM 1(c)
```

```r
cat("E[X] =", round(expected_X, 4), "\n\n")
```

```
## E[X] = 1.3333
```

2. The random variable X is said to have the Yule-Simon distribution if

P(X=n) = 4/n(n+1)(n+2)' n>=1

(a) Show that the PMF sums to 1.

Sum P(X = n), from n = 1 to infinity:

= 4 * Sum [1 / (n(n+1)(n+2))]

Using partial fractions:

1/[n(n+1)(n+2)]

= (1/2)[1/(n(n+1)) - 1/((n+1)(n+2))]

Therefore:

Sum P(X = n)

= 2 * Sum [1/(n(n+1)) - 1/((n+1)(n+2))]

Expanding:

= 2[(1/(12) - 1/(23))

+ (1/(23) - 1/(34))

+ (1/(34) - 1/(45)) + ...]

By telescoping:

= 2[1/2 - 1/((N+1)(N+2))]

Taking the limit as N approaches infinity:

= 2[1/2 - 0]

= 1

FINAL ANSWER:

```r
# Define the probability mass function.

yule_simon_pmf <- function(n) {
  4 / (n * (n + 1) * (n + 2))
}


# (a) Calculate the finite partial sum of the PMF.

N <- 10000

n_values <- 1:N

partial_pmf_sum <- sum(yule_simon_pmf(n_values))

cat("PROBLEM 2(a)\n")
```

```
## PROBLEM 2(a)
```

```r
cat("Partial sum of PMF =", round(partial_pmf_sum, 4), "\n")
```

```
## Partial sum of PMF = 1
```

```r
cat("Theoretical infinite sum = 1.0000\n\n")
```

```
## Theoretical infinite sum = 1.0000
```

```r
# (b) Calculate the finite partial expected value.

partial_mean <- sum(n_values * yule_simon_pmf(n_values))

cat("PROBLEM 2(b)\n")
```

```
## PROBLEM 2(b)
```

```r
cat("Partial expected value =", round(partial_mean, 4), "\n")
```

```
## Partial expected value = 1.9996
```

```r
cat("Theoretical E[X] = 2.0000\n\n")
```

```
## Theoretical E[X] = 2.0000
```

```r
# (c) Calculate partial sums of the second moment.

# R can only calculate finite sums.
# Increasing N demonstrates the divergence of the series.

N_values <- c(10, 100, 1000, 10000)
```

```r
partial_second_moments <- sapply(
  N_values,
  function(N_current) {
    n <- 1:N_current
    sum(n^2 * yule_simon_pmf(n))
  }
)

cat("PROBLEM 2(c)\n")
```

```
## PROBLEM 2(c)
```

```r
print(data.frame(
  N = N_values,
  Partial_Second_Moment = round(partial_second_moments, 4)
))
```

```
##       N Partial_Second_Moment
## 1    10                4.7462
## 2   100               12.8675
## 3  1000               21.9539
## 4 10000               31.1516
```

```r
cat("Theoretical E[X^2] = Inf\n\n")
```

```
## Theoretical E[X^2] = Inf
```

3. The probability density function of X is given by

GIVEN:

f(x) = a + bx^2, for 0 <= x <= 1

f(x) = 0, otherwise

E[X] = 3/5

Find a and b.

Step 1: Use the total probability condition.

Integral from 0 to 1 of f(x) dx = 1

Integral from 0 to 1 of (a + bx^2) dx = 1

a Integral from 0 to 1 of dx

+ b Integral from 0 to 1 of x^2 dx = 1

a[x] from 0 to 1 + b[x^3/3] from 0 to 1 = 1

a(1 - 0) + b(1/3 - 0) = 1

a + b/3 = 1

Multiply both sides by 3:

3a + b = 3 (Equation 1)

Step 2: Use the expected value condition.

E[X] = Integral from 0 to 1 of x*f(x) dx

3/5 = Integral from 0 to 1 of x(a + bx^2) dx

3/5 = Integral from 0 to 1 of (ax + bx^3) dx

3/5 = a[x^2/2] from 0 to 1

+ b[x^4/4] from 0 to 1

3/5 = a(1/2) + b(1/4)

```r
# Write the two simultaneous equations in matrix form:
# a + b/3 = 1
# a/2 + b/4 = 3/5

A <- matrix(
  c(1, 1/3,
    1/2, 1/4),
  nrow = 2,
  byrow = TRUE
)

rhs <- c(1, 3/5)

# Solve the simultaneous equations.

ab <- solve(A, rhs)

a <- unname(ab[1])
b <- unname(ab[2])

cat("PROBLEM 3\n")
```

```
## PROBLEM 3
```

```r
cat("a =", round(a, 4), "\n")
```

```
## a = 0.6
```

```r
cat("b =", round(b, 4), "\n\n")
```

```
## b = 1.2
```

```r
# Verify the total probability and expected value.

f <- function(x) {
  ifelse(x >= 0 & x <= 1, a + b*x^2, 0)
}

total_probability <- integrate(
  f,
  lower = 0,
  upper = 1
)$value

expected_value_check <- integrate(
  function(x) x*f(x),
  lower = 0,
  upper = 1
)$value

cat("VERIFICATION\n")
```

```
## VERIFICATION
```

```
cat("Total probability =", round(total_probability, 4), "\n")
```

```
## Total probability = 1
```

```
cat("E[X] =", round(expected_value_check, 4), "\n\n")
```

```
## E[X] = 0.6
```

4. Let X be a random variable having expected value mu and variance sigma2. Find the expected value and variance of

Y = X-mu/sigma

GIVEN:

E[X] = mu

Var(X) = sigma^2

Y = (X - mu)/sigma

Find E[Y] and Var(Y)

(a) Find E[Y].

Given:

Y = (X - mu)/sigma

E[Y] = E[(X - mu)/sigma]

= (1/sigma)E[X - mu]

= (1/sigma)[E[X] - mu]

Since E[X] = mu:

= (1/sigma)(mu - mu)

= (1/sigma)(0)

= 0

FINAL ANSWER:

E[Y] = 0.0000

(b) Find Var(Y).

Given:

Y = (X - mu)/sigma

```r
## The results follow directly from the given formulas.

expected_Y <- 0
variance_Y <- 1

cat("PROBLEM 4(a)\n")
```

```
## PROBLEM 4(a)
```

```r
cat("E[Y] =", round(expected_Y, 4), "\n\n")
```

```
## E[Y] = 0
```

```r
cat("PROBLEM 4(b)\n")
```

```
## PROBLEM 4(b)
```

```r
cat("Var(Y) =", round(variance_Y, 4), "\n\n")
```

```
## Var(Y) = 1
```

5. Let X and Y be independent Poisson random variables with parameters lambda1 = 2 and lambda2 = 3, respectively. Let W = X + Y.

(a) Using the property of sums of independent random variables, derive the moment generating function (MGF) MW (t) of W.

(b) Based on the uniqueness property of MGFs, identify the probability distribution and parameter(s) of W.

(c) Determine the expected value E[W] and the second moment E[W 2] by successively differentiating MW (t) and evaluating it at t = 0.

(a) Derive the MGF of W.

The MGF of a Poisson random variable is:

M(t) = exp(lambda * (exp(t) - 1))

For X ~ Poisson(2):

M_X(t) = exp(2(exp(t) - 1))

For Y ~ Poisson(3):

M_Y(t) = exp(3(exp(t) - 1))

Since X and Y are independent:

M_W(t) = M_X(t) * M_Y(t)

= exp(2(exp(t) - 1)) * exp(3(exp(t) - 1))

= exp(2(exp(t) - 1) + 3(exp(t) - 1))

= exp((2 + 3)(exp(t) - 1))

```r
# Define the Poisson parameters.

lambda1 <- 2
lambda2 <- 3

# Since X and Y are independent Poisson random variables,
# the parameter of W is the sum of their parameters.

lambda_W <- lambda1 + lambda2


# (a) Display the derived MGF.

cat("PROBLEM 5(a)\n")
```

```
## PROBLEM 5(a)
```

```r
cat("M_W(t) = exp(5 * (exp(t) - 1))\n\n")
```

```
## M_W(t) = exp(5 * (exp(t) - 1))
```

```r
# Define the MGF symbolically.

M_W <- expression(
  exp(lambda_W * (exp(t) - 1))
)


# Differentiate the MGF once.

first_derivative <- D(M_W, "t")

cat("First derivative M'_W(t):\n")
```

```
## First derivative M'_W(t):
```

```r
print(first_derivative)
```

```
## exp(lambda_W * (exp(t) - 1)) * (lambda_W * exp(t))
```

```r
# Differentiate the MGF a second time.

second_derivative <- D(first_derivative, "t")

cat("\nSecond derivative M''_W(t):\n")
```

```
##
## Second derivative M''_W(t):
```

```r
print(second_derivative)
```

```
## exp(lambda_W * (exp(t) - 1)) * (lambda_W * exp(t)) * (lambda_W *
##     exp(t)) + exp(lambda_W * (exp(t) - 1)) * (lambda_W * exp(t))
```

```r
# (b) Identify the distribution of W.

cat("\nPROBLEM 5(b)\n")
```

```
##
## PROBLEM 5(b)
```

```r
cat("W ~ Poisson(", lambda_W, ")\n\n")
```

```
## W ~ Poisson( 5 )
```

```r
# (c) Evaluate the first and second derivatives at t = 0.

# E[W] = M'_W(0)

expected_W <- eval(
  first_derivative,
  envir = list(t = 0, lambda_W = lambda_W)
)

# E[W^2] = M''_W(0)

second_moment_W <- eval(
  second_derivative,
  envir = list(t = 0, lambda_W = lambda_W)
)

cat("PROBLEM 5(c)\n")
```

```
## PROBLEM 5(c)
```

```r
cat("E[W] =", round(expected_W, 4), "\n")
```

```
## E[W] = 5
```

```r
cat("E[W^2] =", round(second_moment_W, 4), "\n\n")
```

```
## E[W^2] = 30
```

```r
# Verify using the Poisson moment formulas.
# E[W] = lambda
# E[W^2] = lambda^2 + lambda

cat("VERIFICATION\n")
```

```
## VERIFICATION
```

```r
cat("E[W] =", lambda_W, "\n")
```

```
## E[W] = 5
```

```r
cat("E[W^2] =", lambda_W^2 + lambda_W, "\n\n")
```

```
## E[W^2] = 30
```